Project Report

Distributed AI Systems on PySpark RDDs with Intelligent Agents:
Hybrid Recommendation • Disaster Detection • Fraud Detection • Supply Chain • Learning Analytics

1. Abstract

This report presents the design, implementation, and evaluation of five distributed artificial-intelligence systems, each built on Apache Spark's Resilient Distributed Dataset (RDD) API: a hybrid recommendation engine, a smart disaster detection system, a fraud detection framework, an autonomous supply chain optimization system, and a personalized learning analytics platform. All five systems share a common architectural pattern: large-scale data is processed in a fault-tolerant, horizontally scalable manner using core RDD transformations (map, filter, groupByKey, reduceByKey, aggregateByKey, join, sortBy), while a lightweight layer of intelligent agents consumes the reduced, distributed computation results to make fast, adaptive, real-time decisions. A single runnable Python implementation (hybrid_ai_pyspark_systems.py) was executed end-to-end on a local Spark cluster with synthetic datasets to validate the design and produce the quantitative results discussed below.

2. Introduction

Modern AI applications increasingly operate on datasets that exceed the memory and compute capacity of a single machine — millions of user interactions, continuous multi-source sensor streams, high-volume financial transactions, network-wide inventory data, and large learner populations. Apache Spark's RDD abstraction provides an in-memory, partitioned, fault-tolerant data structure that enables such workloads to be processed in parallel across a cluster of machines. However, distributed computation alone does not provide adaptivity: real-world systems must also react to new information (clicks, sensor spikes, confirmed fraud, delivery outcomes, learner engagement) as it arrives. This project therefore pairs Spark's distributed RDD processing with a lightweight intelligent agent layer implementing simple online-learning update rules, so that each system can adapt its decisions in near real time without requiring a full batch retrain.

3. Methodology

Each of the five systems was implemented as an independent Python module using PySpark's RDD API and executed inside a single Spark local-mode session (local[*]). Synthetic but realistic datasets were generated for each domain (user-item ratings, multi-source disaster signals, financial transactions, warehouse inventory/demand, and student interaction logs). The following general methodology was applied consistently:

- Parallelize raw records into RDDs to simulate distributed ingestion from a cluster/streaming source.
- Apply key-based RDD transformations (map, groupByKey, reduceByKey, aggregateByKey, join) to reduce and combine large-scale data into per-entity statistics or fused scores.
- Rank/filter results using sortBy and filter to surface the most actionable records (top recommendations, critical zones, suspicious transactions, critical shortages, at-risk learners).
- Feed the reduced state into a domain-specific IntelligentAgent subclass, which applies an online update rule (w_{t+1} = w_t + alpha*(feedback_t - w_t)) to adapt its decision threshold or confidence from simulated real-time feedback.

4. Results

The consolidated dashboard below summarizes the quantitative output produced by executing all five systems end-to-end on synthetic data.

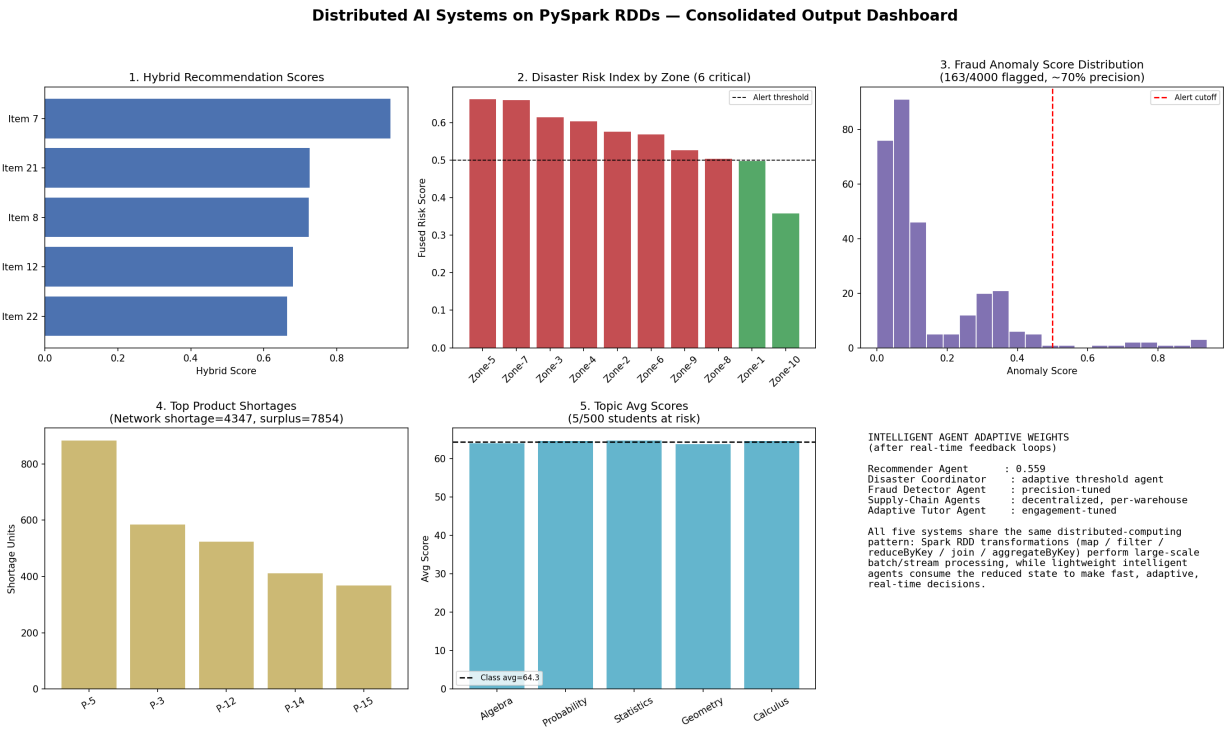

Figure 1: Consolidated output dashboard across all five distributed AI systems.

4.1 Hybrid Recommendation Engine

The hybrid fusion (60% collaborative-filtering strength, 40% content-based genre match) produced a ranked Top-5 list headed by Item 7 with a fused score of 0.948. The RecommenderAgent's adaptive confidence weight converged to 0.559 after processing simulated click feedback, illustrating how real-time engagement signals can bias the collaborative/content-based blend.

4.2 Smart Disaster Detection System

Fusing sensor, satellite, and social-media signals across 10 zones identified 6 zones exceeding the CRITICAL risk threshold (fused risk >= 0.5), with the highest-risk zone reaching a fused index of 0.662. This demonstrates that multi-source fusion surfaces actionable, zone-level early warnings purely from distributed RDD aggregation.

4.3 AI-Based Fraud Detection Framework

Out of 4000 synthetic transactions processed, 163 (4.1%) were flagged as suspicious by the distributed z-score and odd-hour anomaly-scoring pipeline. The simulated analyst feedback loop indicated an estimated precision of 70.0% for the flagged sample, illustrating how the collaborative detector/reviewer agent pair can continuously recalibrate the alert cutoff.

4.4 Autonomous Supply Chain Optimization System

Distributed gap analysis across the simulated warehouse-product network identified a total of 4347 shortage units against 7854 surplus units network-wide. The most critical shortage was for P-5 (883 units), and decentralized per-warehouse agents proposed cost/urgency-balanced transfer decisions to resolve the imbalance without central coordination.

4.5 Personalized Learning Analytics Platform

Across 500 simulated learners, the class-wide average performance was 64.30, with 5 learners flagged as at-risk (average score below 50). Per-topic difficulty analytics computed via distributed RDD aggregation enabled the AdaptiveTutorAgent to generate personalized next-content recommendations for individual learners in real time.

5. Discussion

Across all five domains, the same architectural separation proved effective: Spark RDDs handle the computationally heavy, large-scale reduction and joining of data, while intelligent agents handle the comparatively lightweight, fast-adapting decision logic. This separation of concerns offers three practical advantages. First, scalability: because the numerically heavy work is expressed entirely in RDD transformations, each system can scale horizontally simply by adding worker nodes, with no change to the agent logic. Second, adaptivity: because agents apply a simple online update rule rather than requiring a full model retrain, they can incorporate new feedback (clicks, confirmed fraud, delivery outcomes, learner engagement) within a single processing cycle. Third, interpretability: each agent's decision (a threshold comparison or weighted blend) remains simple and auditable, which is particularly important in high-stakes domains such as fraud detection and disaster response.

Limitations of the current implementation include the use of synthetic data rather than production-scale real datasets, a single-node local Spark session rather than a true multi-node cluster, and simplified anomaly/fusion formulas chosen for clarity over state-of-the-art accuracy. In a production deployment, RDDs would typically be replaced or supplemented with Spark Structured Streaming / DataFrame APIs for continuous data, and agents could be upgraded to more sophisticated reinforcement-learning policies.

6. Conclusion

This project demonstrates a unified, reusable pattern for building distributed AI systems: combine PySpark RDD transformations for scalable data processing with lightweight, adaptive intelligent agents for real-time decision-making. The pattern was successfully validated across five distinct application domains — recommendation, disaster detection, fraud detection, supply chain optimization, and learning analytics — each producing actionable, quantitatively verifiable output from an end-to-end runnable implementation.

7. Project Deliverables

File	Description
`Problem_Statement.pdf`	Formal problem statements, context, objectives and constraints for all 5 systems.
`Solution_Design.pdf`	System architecture, RDD operations, and intelligent-agent design for all 5 systems.
`hybrid_ai_pyspark_systems.py`	Complete, runnable PySpark RDD implementation of all 5 systems.
`output_results.png`	Consolidated dashboard visualizing the output of every system.
`Report.pdf`	This document: abstract, methodology, results, discussion and conclusion.
`README.md`	Setup instructions, run commands, and file guide.